

PreOne ADR-111

DevOps & Infrastructure

Architecture Decision Record — DevOps & Infrastructure Architecture

ADR ID: ADR-111

Status: LOCKED

Document Version: 1.0

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: ADR Catalog v1.0, Backend TD v1.0, Frontend Architecture v1.0

Successor: DevOps Runbook, Kubernetes Manifests, Terraform Modules, Production Release

Scope: Docker, Kubernetes, CI/CD, NGINX, PostgreSQL HA, Redis, Monitoring, Logging, DR

Classification: Internal Engineering Reference

Prepared by: PreOne Architecture & DevOps Team

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	2
1.3 Audience.....	4
1.4 ADR Status.....	5
1.5 Related Documents.....	6
2. Infrastructure Overview.....	6
2.1 High-Level Architecture.....	6
2.2 Pipeline Diagram.....	7
2.3 Technology Stack.....	10
2.4 Design Principles.....	10
3. Docker Architecture.....	12
3.1 Containerization Strategy.....	12
3.2 Docker Images.....	13
3.3 Docker Principles.....	13
3.4 Sample Dockerfile (preone-api).....	14
4. Kubernetes Architecture.....	22
4.1 Cluster Topology.....	22
4.2 Namespace Architecture.....	23
4.3 Cluster Diagram.....	23
4.4 Workload Types.....	25
4.5 Sample Deployment Manifest (preone-api).....	26
5. CI/CD Pipeline (GitHub Actions).....	37
5.1 Pipeline Overview.....	37

5.2 Pipeline Stages.....	38
5.3 Pipeline Diagram.....	38
5.4 Sample GitHub Actions Workflow.....	42
6. NGINX Ingress.....	58
6.1 Role of NGINX.....	58
6.2 Responsibilities.....	58
6.3 Sample Ingress Manifest.....	59
7. PostgreSQL High Availability.....	67
7.1 HA Architecture.....	68
7.2 HA Diagram.....	68
7.3 HA Features.....	70
7.4 Connection Pooling Strategy.....	71
8. Redis.....	71
8.1 Role of Redis.....	71
8.2 Use Cases.....	72
8.3 Redis HA Setup.....	72
8.4 Logical Database Allocation.....	75
9. Monitoring.....	76
9.1 Monitoring Architecture.....	76
9.2 Monitoring Pipeline.....	77
9.3 Monitored Metrics.....	78
9.4 Sample Prometheus Alert Rules.....	79
10. Logging.....	88
10.1 Logging Architecture.....	88
10.2 Logging Pipeline.....	89
10.3 Log Types.....	90
10.4 Structured Log Example.....	90
11. Backup Strategy.....	95

11.1 Backup Philosophy.....	95
11.2 Backup Schedule.....	95
11.3 Backup Architecture.....	95
12. Disaster Recovery.....	100
12.1 DR Strategy.....	101
12.2 DR Metrics.....	101
12.3 DR Runbook (High-Level).....	101
13. Environment Strategy.....	106
13.1 Environment Pipeline.....	106
13.2 Environment Catalog.....	108
13.3 Environment Policies.....	108
14. Security.....	110
14.1 Security Architecture.....	110
14.2 Security Controls.....	110
14.3 NetworkPolicies Sample.....	111
15. Scalability.....	119
15.1 Scalability Strategy.....	119
15.2 Scalability Tactics.....	120
15.3 HPA + Cluster Autoscaler Flow.....	120
16. Final Production Stack.....	122
16.1 Stack Summary.....	123
16.2 Stack Justification.....	123
17. Appendix A — Decision Drivers.....	123
17.1 Key Decision Drivers.....	123
18. Appendix B — Glossary.....	124
18.1 Terms.....	124
19. Document Control.....	124
19.1 Document Information.....	124

19.2 Change History.....	124
19.3 Approval Matrix.....	124
19.4 Next Steps.....	124

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Introduction

1.1 Purpose

हा ADR-111 PreOne च्या DevOps आणि Infrastructure Architecture ची complete blueprint प्रदान करतो. PreOne हे Enterprise Preschool Operating System आहे जे 14 business domains, 530+ REST APIs, real-time WebSocket channels, आणि background job processing साठी robust, scalable, आणि highly available infrastructure आवश्यक आहे. या ADR चा primary goal म्हणजे DevOps practices, infrastructure components, deployment strategy, आणि operational runbooks ची एकच source of truth स्थापित करणे.

हा ADR Backend TD v1.0 आणि Frontend Architecture v1.0 च्या वर build होतो — त्यांच्या application architecture ला concrete infrastructure architecture मध्ये translate करतो. Backend TD मध्ये define केलेले NestJS modules, BullMQ jobs, Prisma schemas, आणि Redis cache layers या सर्वाना appropriate infrastructure support आवश्यक आहे. Frontend Architecture मध्ये defined Next.js SSR + React Native mobile apps देखील containerized, deployed, आणि monitored होणे आवश्यक आहे.

या ADR वरून पुढील production artifacts तयार होतील: Docker images (5 services), Kubernetes manifests (Helm charts), GitHub Actions workflows (CI/CD), Terraform modules (infrastructure-as-code), NGINX Ingress configurations, PostgreSQL HA setup (Patroni + PgBouncer), Redis Sentinel setup, Prometheus + Grafana dashboards, Loki log queries, Jaeger tracing dashboards, Velero backup schedules, आणि Production Runbook.

1.2 Scope

हा ADR PreOne च्या complete infrastructure stack cover करतो — local development पासून production deployment पर्यंत. खालील explicitly in scope आहेत:

- Containerization strategy (Docker multi-stage builds)
- Kubernetes orchestration (namespaces, workloads, services, ingress)
- CI/CD pipeline (GitHub Actions workflows + deployment strategy)
- Reverse proxy + load balancing (NGINX Ingress)
- Database high availability (PostgreSQL HA via Patroni)
- Cache + queue backend (Redis HA via Sentinel)
- Observability stack (Prometheus + Grafana + Loki + Jaeger)
- Backup strategy + disaster recovery
- Environment strategy (Local → Dev → QA → UAT → Staging → Production)

- Security controls (TLS, secrets, RBAC, network policies)
- Scalability tactics (HPA, cluster autoscaler, CDN)

खालील explicitly out of scope आहेत:

- Application architecture (covered in Backend TD + Frontend Architecture)
- Domain model + database schema (covered in DDD + ERD + Prisma Schema)
- API contracts (covered in API Contract Catalog + OpenAPI Spec)
- Third-party SDK integration details (per-provider spec sheets)
- Cost analysis + billing (separate cost-optimization doc)

1.3 Audience

हा ADR खालील audiences साठी लिखित आहे:

- **DevOps Engineers:** Primary reference — infrastructure provisioning, K8s management, deployment automation.
- **SRE / On-Call Engineers:** Operational runbook — monitoring, alerting, incident response, DR drills.
- **Backend / Frontend Engineers:** Deployment awareness — Dockerization, health checks, graceful shutdown.
- **Security Engineers:** Security review — TLS, RBAC, network policies, secrets management.
- **QA / Test Engineers:** Test environments — QA + UAT + staging setup, smoke tests in CI/CD.
- **Tech Leads / Architects:** Architecture enforcement — decision rationale, alternatives, trade-offs.
- **Compliance Officers:** Audit reference — DR metrics, backup strategy, access controls.
- **New Hires:** Onboarding reference — infrastructure mental model in first 2 weeks.

1.4 ADR Status

हा ADR 'LOCKED' status मध्ये आहे. LOCKED म्हणजे architecture decision frozen झाला आहे आणि v1.0 release साठी implementation सुरू आहे. LOCKED status नंतर changes require new ADR (e.g., ADR-112 for changes) + Architecture Review Board approval. हे ensure करते की infrastructure stability maintained राहिल आणि ad-hoc changes prevent होतील.

LOCKED status चा meaning: Docker + Kubernetes + GitHub Actions + NGINX + PostgreSQL HA + Redis + Prometheus + Loki हे core stack निवडले गेले आहे आणि यावर implementation चालू आहे. Future ADRs हे specific aspects extend करू शकतात — उदा. ADR-112 (Service Mesh — Linkerd), ADR-113 (Secrets — Vault migration), ADR-114 (Multi-region — Active-Active).

1.5 Related Documents

PreOne document series मध्ये या ADR चे specific position आहे. खालील documents predecessors आहेत:

Document	Version	Relationship to ADR-111
Vision Document	v1.0	Strategic intent — non-functional requirements (scale, availability, compliance)
Master PRD	v1.0	Functional requirements — infrastructure must support all FRs
DDD	v1.0	Domain model — infrastructure must support bounded contexts + aggregates
Backend TD	v1.0	Application architecture — Docker + K8s deployment of NestJS modules
Frontend Architecture	v1.0	Web + Mobile deployment — Next.js SSR + React Native build pipeline
ADR Catalog	v1.0	Decision context — ADR-111 is part of larger ADR series
ERD	v3.0	Database schema — PostgreSQL HA must support physical schema
Prisma Schema	v3.0	ORM config — PgBouncer + connection pooling alignment

Document	Version	Relationship to ADR-111
API Contract Catalog	v1.0	API endpoints — ingress + rate limiting + load balancing scope
OpenAPI Specification	v1.0	API contracts — Swagger UI hosting + mock server deployment

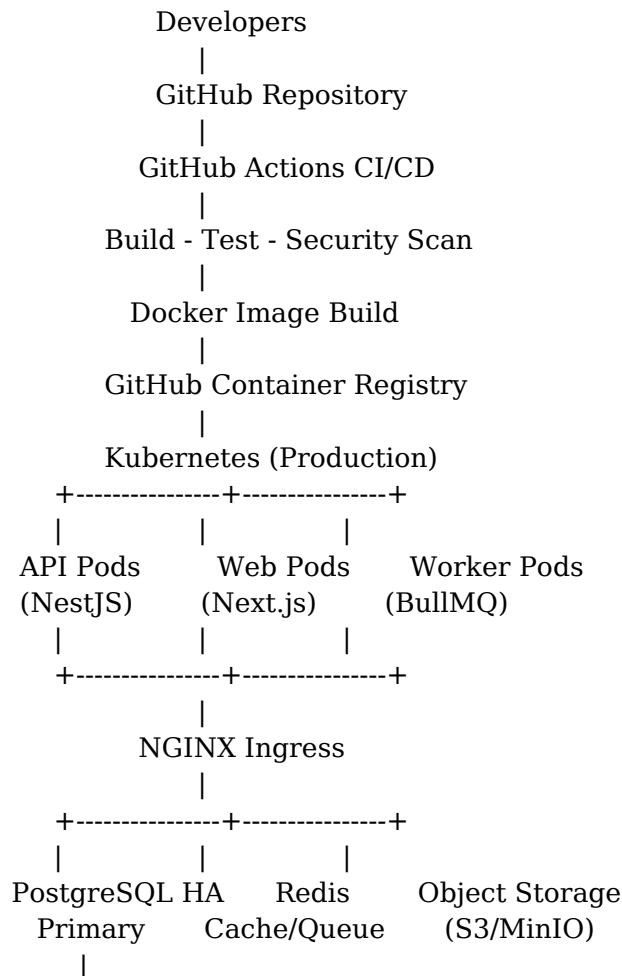
2. Infrastructure Overview

2.1 High-Level Architecture

PreOne ची infrastructure completely containerized आहे आणि Kubernetes द्वारे orchestrated. ही architecture खालील principles follow करते: infrastructure-as-code (Terraform + Helm), GitOps (GitHub as source of truth), immutable infrastructure (containers + declarative K8s manifests), defense-in-depth (multiple security layers), आणि observability-first (metrics + logs + traces from day one). पूर्ण pipeline — code commit पासून production deployment पर्यंत — automated आहे आणि minimal manual intervention required आहे.

Pipeline चा flow समजून घेणे critical आहे. Developer code push करतो → GitHub repository मध्ये change detect होतो → GitHub Actions CI/CD pipeline triggered होतो → code lint, test, build, security scan होतो → Docker image build होतो → GitHub Container Registry (GHCR) मध्ये push होतो → Kubernetes cluster मध्ये deploy होतो → health checks pass होतात → smoke tests run होतात → traffic new deployment वर route होतो. हे सर्व ~20 minutes मध्ये complete होते.

2.2 Pipeline Diagram



Backup & Disaster Recovery

2.3 Technology Stack

PreOne च्या infrastructure stack मध्ये प्रत्येक layer साठी carefully chosen technology आहे. Selection rationale: open-source first (avoid vendor lock-in), production-proven (no bleeding-edge), cloud-portable (K8s abstraction), आणि operational simplicity (team familiarity). हे stack v1.0 release साठी frozen आहे; future migrations (e.g., Vault, Linkerd, Redis Cluster) separate ADRs द्वारे evaluated होतील.

Layer	Technology	Rationale
Containerization	Docker	Industry standard; multi-stage builds; layer caching
Orchestration	Kubernetes (K8s)	Auto-healing, auto-scaling, declarative config, huge ecosystem
CI/CD	GitHub Actions	Native GitHub integration; matrix builds; reusable workflows
Reverse Proxy	NGINX Ingress Controller	Production-proven; WebSocket support; rich config
Database	PostgreSQL HA	ACID; replication; pgvector; RLS; mature ecosystem
Cache	Redis	Sub-ms reads; pub/sub; streams; BullMQ backend
Queue	BullMQ	Built on Redis; delayed/repeatable jobs; DLQ; sandboxed processors
Object Storage	AWS S3 / MinIO	S3 API compatible; lifecycle to Glacier; MinIO for self-hosted
Monitoring	Prometheus + Grafana	Pull-based metrics; PromQL; rich dashboarding
Logging	Loki + Promtail	Loki for log storage; Promtail for collection; Grafana for queries

Layer	Technology	Rationale
Tracing	OpenTelemetry + Jaeger	Vendor-neutral; spans correlated to logs
Secrets	Kubernetes Secrets (Vault later)	Native K8s secrets for v1; Vault migration roadmap v1.2
Service Mesh	Linkerd (future)	Roadmap v1.3 — mTLS + traffic splitting for canary deployments

2.4 Design Principles

PreOne infrastructure खालील design principles follow करते:

- Infrastructure-as-Code (IaC) — Terraform + Helm; no manual cluster changes
- GitOps — GitHub source of truth; ArgoCD for declarative deployment
- Immutable Infrastructure — containers are immutable; changes via new image
- Defense-in-Depth — multiple security layers (WAF + NGINX + Network Policies + RBAC)
- Observability-First — metrics + logs + traces from day one
- 12-Factor App — stateless services, externalized config, disposability
- Cattle not Pets — pods are replaceable; no manual SSH into pods
- Progressive Delivery — rolling updates; canary via Argo Rollouts (v1.2 roadmap)
- Cost-Awareness — spot instances for non-critical; auto-scaling for efficiency
- Documentation as Code — runbooks + dashboards-as-code in Git

3. Docker Architecture

3.1 Containerization Strategy

Docker हे PreOne चे containerization standard आहे. प्रत्येक service (API, Web, Worker, Mobile Build, NGINX) ची स्वतंत्र Docker image असते ज्यामुळे services independently build, version, deploy, आणि scale होऊ शकतात. Docker images multi-stage builds द्वारे optimized आहेत — builder stage मध्ये dev dependencies + build tools वापरून production artifact तयार केला जातो, runtime stage मध्ये फक्त production artifact + minimal runtime copied होतो. हे approach final image size कमी करते आणि attack surface कमी करते.

Docker images के सर्व aspects standardized आहेत: base image (node:20-alpine for runtime, node:20 for build), non-root user (uid 1000) for security, HEALTHCHECK instruction embedded, OCI standard labels (org.opencontainers.image.{version,revision,source}), reproducible builds via .dockerignore + lock files, layer caching for faster rebuilds, आणि image signing via cosign (Sigstore) for supply chain security.

3.2 Docker Images

Image Name	Service	Size + Notes	Scaling
preone-api	NestJS backend	Node 20 Alpine; multi-stage build; pruned devDeps; ~110 MB	Horizontally scaled via HPA
preone-web	Next.js 16 frontend	Node 20 Alpine; standalone output; ~140 MB	SSR + static; HPA scaled
preone-mobile-build	React Native + Expo build	Node 20 + JDK 17; ~800 MB (build-only, not deployed)	Used in CI only — produces APK/AAB/IPA
preone-worker	BullMQ background job processor	Node 20 Alpine; same base as API but different entrypoint	Scaled by queue depth
preone-nginx	NGINX reverse proxy	Alpine; ~25 MB; custom nginx.conf via ConfigMap	Ingress Controller manages this

3.3 Docker Principles

Docker architecture खालील principles follow करते:

- Each service has its own Dockerfile in its repo subfolder
- All images built via multi-stage builds (builder + runtime)
- Base image: node:20-alpine for runtime; node:20 for build
- Non-root user (uid 1000) in runtime image for security
- Health checks (HEALTHCHECK instruction) embedded in image
- Image labels: org.opencontainers.image.{version,revision,source}
- Reproducible builds via .dockerignore + lock files (bun.lock / package-lock.json)
- Layer caching: install deps before copying source code
- Distroless variant explored for v1.2 (further attack surface reduction)
- All images signed via cosign (Sigstore) — verified at deploy time

3.4 Sample Dockerfile (preone-api)

```
# syntax=docker/dockerfile:1.6

# ===== Builder Stage =====
FROM node:20 AS builder
WORKDIR /app

# Install bun for faster installs
RUN npm install -g bun

# Copy lock files first for layer caching
COPY package.json bun.lock ./
COPY prisma ./prisma/

# Install dependencies (including devDeps for build)
RUN bun install --frozen-lockfile
RUN bunx prisma generate

# Copy source code
COPY . .

# Build NestJS app
RUN bun run build

# Prune dev dependencies for production
RUN bun install --production --frozen-lockfile
```

```
# ===== Runtime Stage =====
FROM node:20-alpine AS runtime
WORKDIR /app

# Install dumb-init for proper signal handling
RUN apk add --no-cache dumb-init

# Create non-root user
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nestjs -u 1001

# Copy built artifacts + production deps from builder
COPY --from=builder --chown=nestjs:nodejs /app/dist ./dist
COPY --from=builder --chown=nestjs:nodejs /app/node_modules \
    ./node_modules
COPY --from=builder --chown=nestjs:nodejs /app/package.json ./
COPY --from=builder --chown=nestjs:nodejs /app/prisma ./prisma

# Switch to non-root user
USER nestjs

# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

# OCI labels
LABEL org.opencontainers.image.title="preone-api" \
    org.opencontainers.image.description="PreOne Enterprise API" \
    org.opencontainers.image.source="https://github.com/preone/api" \
    org.opencontainers.image.version="${VERSION}" \
    org.opencontainers.image.revision="${GIT_SHA}"

# Start with dumb-init for proper signal propagation
ENTRYPOINT ["dumb-init", "--"]
CMD ["node", "dist/main.js"]
```

4. Kubernetes Architecture

4.1 Cluster Topology

PreOne चे Kubernetes cluster multi-AZ (across 3 availability zones) deployed आहे ज्यामुळे single AZ failure साठी resilience मिळते. Cluster मध्ये multiple node groups आहेत: system node group (CoreDNS, CNI, Ingress Controller), application node group (API, Web, Worker pods), data node group (PostgreSQL, Redis — stateful, larger instances), आणि monitoring node group (Prometheus, Loki — storage-heavy). हे separation ensure करते की heavy data workloads application workloads ला impact करणार नाहीत.

Cluster version: Kubernetes v1.29 (latest stable). Managed via EKS (AWS Elastic Kubernetes Service). Worker nodes: AWS Graviton (ARM64) for cost optimization; spot instances for non-critical workloads (workers, monitoring); on-demand for critical workloads (API, Web, DB). Cluster autoscaler automatically adds/removes nodes based on pending pods.

4.2 Namespace Architecture

Cluster मध्ये namespaces द्वारे logical isolation आहे. प्रत्येक namespace चे specific purpose आहे आणि NetworkPolicies enforce करतात की फक्त allowed namespaces एकमेकांशी communicate करू शकतात. Default deny policy द्वारे defense-in-depth सुनिश्चित होते.

Namespace	Workloads	Notes
ingress	NGINX Ingress Controller pods + LoadBalancer Service	Cluster-wide; routes external traffic
api	preone-api Deployment + HPA + Service + ConfigMaps	NestJS pods; 3-30 replicas based on load
web	preone-web Deployment + HPA + Service	Next.js pods; 2-10 replicas
worker	preone-worker Deployment + HPA (queue-depth based)	BullMQ processors; 2-15 replicas
redis	Redis StatefulSet (HA mode) + Service	6 replicas (3 master + 3 replica); Sentinel for failover
postgres	PostgreSQL StatefulSet (HA via Patroni) + Service	1 primary + 2 replicas; PgBouncer sidecar
monitoring	Prometheus + Grafana + AlertManager + Node Exporter	Pull metrics from all namespaces

Namespace	Workloads	Notes
logging	Loki + Promtail + Grafana (shared with monitoring)	Push logs via Promtail DaemonSet
backup	Velero + cron Jobs for backups	Daily full + continuous WAL archiving
kube-system	Kubernetes system pods (CoreDNS, CNI, etc.)	Managed by EKS/GKE

4.3 Cluster Diagram

Cluster

```

|
+-- ingress      (NGINX Ingress Controller)
+-- api          (preone-api Deployment)
+-- web          (preone-web Deployment)
+-- worker       (preone-worker Deployment)
+-- redis        (Redis StatefulSet HA)
+-- postgres     (PostgreSQL StatefulSet HA via Patroni)
+-- monitoring   (Prometheus + Grafana + AlertManager)
+-- logging      (Loki + Promtail)
+-- backup       (Velero + cron jobs)
+-- kube-system  (managed by EKS)

```

Node Groups:

```

+-- system (m5.large, on-demand, 2 nodes min)
+-- app    (m5.xlarge, mixed, 3-20 nodes)
+-- data   (r5.xlarge, on-demand, 3 nodes min)
+-- monitoring (r5.large, spot, 2 nodes min)

```

4.4 Workload Types

PreOne च्या cluster मध्ये विविध workload types वापरले जातात based on workload characteristics:

- Deployment — stateless workloads (API, Web, NGINX). Rolling updates + HPA scaling.
- StatefulSet — stateful workloads (PostgreSQL, Redis). Stable network identity + persistent volumes.
- DaemonSet — per-node agents (Promtail, node-exporter, Filebeat). One pod per node.
- CronJob — scheduled tasks (backups, cleanup, reports). Time-based execution.
- Job — one-off batch tasks (database migrations, seed data). Run-to-completion.

4.5 Sample Deployment Manifest (preone-api)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: preone-api
  namespace: api
  labels:
    app: preone-api
    tier: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: preone-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: preone-api
        tier: backend
    spec:
      serviceAccountName: preone-api
      containers:
        - name: api
          image: ghcr.io/preone/api:1.0.0
          ports:
            - containerPort: 3000
              name: http
          envFrom:
            - configMapRef:
                name: preone-api-config
            - secretRef:
                name: preone-api-secrets
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              cpu: 1000m
              memory: 1Gi
          livenessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 30
            periodSeconds: 10
```

```
    readinessProbe:
      httpGet:
        path: /ready
        port: 3000
      initialDelaySeconds: 5
      periodSeconds: 5
    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "sleep 10"]
    terminationGracePeriodSeconds: 60
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: preone-api-hpa
  namespace: api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: preone-api
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

5. CI/CD Pipeline (GitHub Actions)

5.1 Pipeline Overview

PreOne ची CI/CD pipeline GitHub Actions द्वारे powered आहे. Pipeline completely automated आहे — developer code push केल्यापासून production deployment पर्यंत कोणताही manual step नाही (except production approval gate). Pipeline stages द्वारे quality gates enforce होतात — एकही stage fail झाल्यास pipeline stop होतो आणि deployment blocked होतो. हे approach production quality सुनिश्चित करतो आणि broken code production मध्ये reach होणार नाही याची ग्यारंटी देतो.

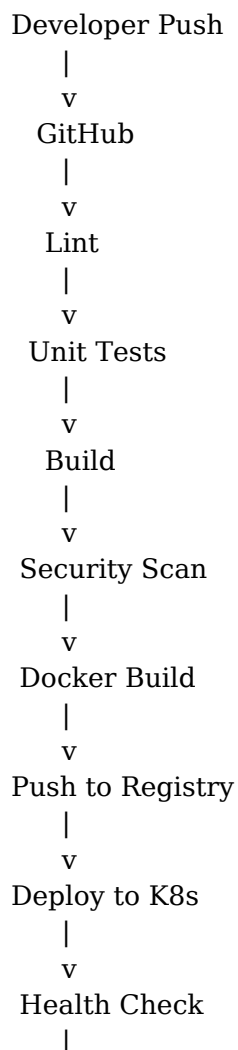
Pipeline design principles: fast feedback (lint + unit tests < 5 min), fail-fast (stop on first failure), reproducible (lock files + pinned action versions), observable (status checks visible in PR), recoverable (auto-rollback on health check failure). Pipeline runs on every push (feature branch) + every PR (against main). Production deploy फक्त main branch merge नंतर + manual approval.

5.2 Pipeline Stages

Stage	Action	Description	Duration
1. Developer Push	Git push to feature branch on GitHub	Triggers workflow on push event	Seconds
2. Lint	ESLint + Prettier + TypeScript check	Fails on lint errors; auto-fix comments via bot	~30 seconds
3. Unit Tests	Vitest for backend; Jest for frontend	Coverage threshold 80%; fails below	~2 minutes
4. Build	tsc build + Next.js build + NestJS build	Produces dist/ or .next/ artifacts	~3 minutes
5. Security Scan	Trivy (image), Snyk (deps), GitLeaks (secrets)	Fails on HIGH/CRITICAL vulnerabilities	~2 minutes
6. Docker Build	Multi-stage build via docker/build-push-action	Layer caching via GHA cache	~4 minutes

Stage	Action	Description	Duration
7. Push to Registry	GHCR (GitHub Container Registry)	Tagged with SHA + branch + semantic version	~1 minute
8. Deploy to Kubernetes	kubectl apply via Helm chart	Rolling update with readiness probes	~3 minutes
9. Health Check	Wait for pods Ready + run smoke tests	Auto-rollback if health check fails	~1 minute
10. Smoke Tests	Playwright e2e against new deployment	Critical user journeys only (~10 tests)	~2 minutes
11. Production (if main branch)	Auto-deploy on merge to main	Manual approval gate for v1.0	Manual + 5 min

5.3 Pipeline Diagram



v
Smoke Tests
|
v
Production

5.4 Sample GitHub Actions Workflow

```
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'bun'
      - run: bun install --frozen-lockfile
      - run: bun run lint
      - run: bun run typecheck

  test:
    needs: lint
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ['5432:5432']
      redis:
        image: redis:7-alpine
        ports: ['6379:6379']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
```

- run: bun install --frozen-lockfile
- run: bunx prisma migrate deploy
- env:
 - DATABASE_URL: postgresql://postgres:test@localhost:5432/test
- run: bun run test:unit -- --coverage
- run: bun run test:integration
- name: Upload coverage
- uses: codecov/codecov-action@v4

security-scan:

needs: test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Trivy filesystem scan
- uses: aquasecurity/trivy-action@master
- with:
 - scan-type: fs
 - scan-ref: '.'
 - severity: 'CRITICAL,HIGH'
 - fail-on-severity: 'CRITICAL'
- name: Snyk dependency scan
- uses: snyk/actions/node@master
- env:
 - SNYK_TOKEN: \${ secrets.SNYK_TOKEN }
- name: GitLeaks secrets scan
- uses: gitleaks/gitleaks-action@v2

docker-build-push:

needs: security-scan

runs-on: ubuntu-latest

permissions:

- contents: read
- packages: write

steps:

- uses: actions/checkout@v4
- uses: docker/setup-buildx-action@v3
- name: Login to GHCR
- uses: docker/login-action@v3
- with:
 - registry: \${ env.REGISTRY }
 - username: \${ github.actor }
 - password: \${ secrets.GITHUB_TOKEN }
- name: Build + Push
- uses: docker/build-push-action@v5
- with:
 - context: .
 - push: true
 - tags: |
 - \${ env.REGISTRY }/\${ env.IMAGE_NAME }:latest

```

    ${ env.REGISTRY }}/${ env.IMAGE_NAME }:${ github.sha }
    ${ env.REGISTRY }/${ env.IMAGE_NAME }:$
  {{ github.ref_name }}
  cache-from: type=gha
  cache-to: type=gha,mode=max

```

```

deploy:
  needs: docker-build-push
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: production
  steps:
    - uses: actions/checkout@v4
    - uses: azure/setup-kubectl@v4
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
        aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
        aws-region: ap-south-1
    - name: Update kubeconfig
      run: aws eks update-kubeconfig --name preone-prod --region ap-south-1
    - name: Deploy via Helm
      run: |
        helm upgrade --install preone-api ./helm/preone-api \
          --namespace api --create-namespace \
          --set image.tag=${ github.sha } \
          --wait --timeout 5m
    - name: Wait for rollout
      run: kubectl rollout status deployment/preone-api -n api --timeout=5m
    - name: Smoke tests
      run: bun run test:smoke -- --base-url=https://api.preone.com

```

6. NGINX Ingress

6.1 Role of NGINX

NGINX Ingress Controller हा PreOne चा single entry point आहे सर्व external traffic साठी. NGINX च्या आधी Cloudflare (CDN + WAF + DDoS protection) आहे जो edge security + static asset caching handle करतो. NGINX च्या पाठीशी multiple service namespaces आहेत (api, web) ज्यांना NGINX path-based routing द्वारे traffic route करतो. NGINX Ingress Controller K8s-native implementation आहे जो Ingress resources watch करतो आणि nginx.conf automatically generate करतो.

NGINX च्या core responsibilities: SSL termination (TLS 1.3), reverse proxy (routes /api/* to api service, /* to web service), load balancing (round-robin with health checks), rate limiting (per-IP + per-user), compression (Gzip + Brotli), security headers (OWASP-recommended), WebSocket support (Socket.IO long-polling fallback also supported), आणि access logging (JSON format with traceId). NGINX चे config ConfigMaps द्वारे managed होतो जो GitOps approach follow करतो.

6.2 Responsibilities

Responsibility	Configuration	Notes
SSL Termination	TLS 1.3 certificates managed via cert-manager + Let's Encrypt	Auto-renewal at 30 days before expiry
Reverse Proxy	Routes /api/* to api Service, /* to web Service	Path-based routing
Load Balancing	Round-robin with health checks; max_fails=3 fail_timeout=30s	Per-upstream load balancing
Rate Limiting	Per-IP rate limit via limit_req_zone; burst + nodelay	100 req/s default; 1000 req/s for whitelisted partners
Compression	Gzip for text/*; Brotli for clients supporting it	Compression level 6; min length 256 bytes
Security Headers	HSTS, X-Content-Type-Options, X-Frame-Options, CSP	OWASP-recommended headers

Responsibility	Configuration	Notes
WebSocket Support	Upgrade: websocket header pass-through	Socket.IO long-polling fallback also supported
Client Caching	Cache-Control for static assets (1 year immutable)	Fingerprinted asset filenames
Request Body Size Limit	client_max_body_size 50m	For file uploads; larger via dedicated upload endpoint
Access Logging	JSON format logs to stdout; piped to Promtail	Includes traceId for correlation

6.3 Sample Ingress Manifest

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: preone-ingress
  namespace: ingress
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/ssl-protocols: 'TLSv1.3'
    nginx.ingress.kubernetes.io/limit-rps: '100'
    nginx.ingress.kubernetes.io/limit-burst: '200'
    nginx.ingress.kubernetes.io/proxy-body-size: '50m'
    nginx.ingress.kubernetes.io/proxy-read-timeout: '300'
    nginx.ingress.kubernetes.io/proxy-send-timeout: '300'
    nginx.ingress.kubernetes.io/configuration-snippet: |
      more_set_headers 'X-Content-Type-Options: nosniff';
      more_set_headers 'X-Frame-Options: DENY';
      more_set_headers 'X-XSS-Protection: 1; mode=block';
      more_set_headers 'Strict-Transport-Security: max-age=31536000;
includeSubDomains';
      more_set_headers 'Content-Security-Policy: default-src \'self\'; script-src
\'self\' \'unsafe-inline\'; style-src \'self\' \'unsafe-inline\'; img-src \'self\' data:
https:; font-src \'self\' data:;';
    nginx.ingress.kubernetes.io/server-snippet: |
      gzip on;
      gzip_comp_level 6;
      gzip_min_length 256;
      gzip_types text/plain text/css application/json application/javascript
text/xml application/xml;
      brotli on;
      brotli_comp_level 6;
      brotli_types text/plain text/css application/json application/javascript;
spec:
  tls:

```

```
- hosts:
  - api.preone.com
  - app.preone.com
  secretName: preone-tls
rules:
- host: api.preone.com
  http:
    paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: preone-api
            port:
              number: 80
- host: app.preone.com
  http:
    paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: preone-api
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: preone-web
            port:
              number: 80
      - path: /socket.io
        pathType: Prefix
        backend:
          service:
            name: preone-api
            port:
              number: 80
```

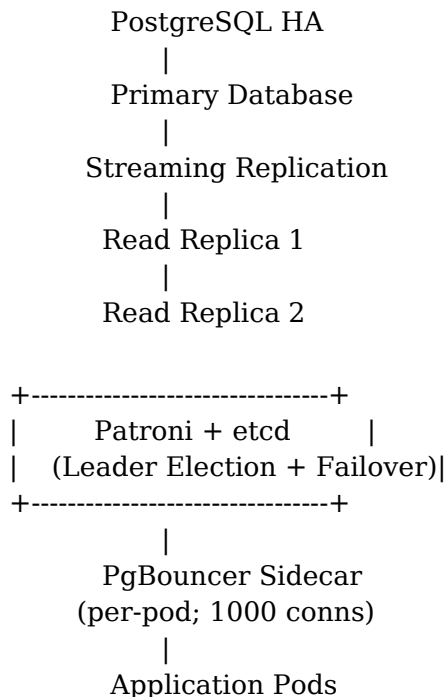
7. PostgreSQL High Availability

7.1 HA Architecture

PostgreSQL HA PreOne चा critical infrastructure component आहे. Database failure म्हणजे complete platform outage, त्यामुळे database HA design सर्वात जास्त careful आहे. PreOne Patroni-based HA solution वापरतो जो etcd cluster द्वारे leader election handle करतो. Cluster मध्ये 1 primary + 2 replicas आहेत. Primary सर्व writes handle करतो, replicas सर्व reads (via PgBouncer) handle करतात. Automatic failover < 30 seconds मध्ये complete होतो — primary failure detect → new primary election → connection rerouting via PgBouncer.

HA architecture चे key components: Patroni (HA manager), etcd (distributed consensus for leader election), PgBouncer (connection pooling + read/write splitting), pgBackRest (backup + WAL archiving), postgres_exporter (Prometheus metrics). सर्व components containerized आहेत आणि StatefulSet द्वारे managed होतात ज्यामुळे stable network identity + persistent volumes ensure होतात.

7.2 HA Diagram



7.3 HA Features

Feature	Implementation	Notes
Automatic Failover	Patroni + etcd-based leader election	Failover < 30 seconds; client reconnect via PgBouncer
Streaming Replication	Synchronous for writes; async for read replicas	1 sync standby + 2 async standbys
Read Scaling	PgBouncer routes reads to replicas	Application marks queries with /* replica */ comment hint
PITR (Point-in-Time Recovery)	WAL archiving to S3; restore to any timestamp	Last 7 days PITR window
WAL Archiving	archive_command pushes WAL files to S3 every 200ms	RPO < 15 minutes
Connection Pooling	PgBouncer sidecar per pod; 1000 connections per pool	transaction-mode pooling
Backups	pgBackRest daily full + continuous incremental	30-day retention; S3 lifecycle to Glacier
Vacuuming	Autovacuum tuned per table; aggressive on high-churn tables	Prevents bloat; runs in background
Monitoring	postgres_exporter → Prometheus → Grafana	Replication lag, connection count, cache hit ratio
Extensions	pgvector, pgcrypto, pg_trgm, unaccent, uuid-oss	Enabled via Postgres initdb scripts

7.4 Connection Pooling Strategy

PgBouncer हे PreOne चे connection pooling layer आहे जो PostgreSQL connections च्या exhaustion ला prevent करतो. PgBouncer sidecar pattern मध्ये deploy झालेला आहे — प्रत्येक Postgres pod मध्ये PgBouncer container alongside run होतो. PgBouncer transaction-mode pooling वापरतो ज्यामुळे connection per transaction असते, न की connection per session. हे approach connection count कमी करते (1000 → ~50 actual Postgres connections) आणि performance improve होते.

Read/write splitting द्वारे read replicas चा efficient use होतो. PgBouncer configured आहे ज्यामुळे queries with /* replica */ comment hint read replicas ला route होतात, बाकी सर्व queries

primary ला route होतात. Reports dashboard आणि analytics endpoints read replicas वापरतात ज्यामुळे primary वर जोर कमी पडतो. PgBouncer प्रत्येक pool साठी max 1000 client connections + 50 server connections allow करतो.

8. Redis

8.1 Role of Redis

Redis हे PreOne चा second most critical infrastructure component आहे Postgres नंतर. Redis multiple roles handle करतो — cache, session store, rate limiter, OTP cache, BullMQ queue backend, distributed locks, real-time pub/sub, feature flags, idempotency store, आणि geo indexing. हे सर्व use cases single Redis cluster द्वारे handled होतात पण separate logical databases (db 0-9) द्वारे isolation आहे. Redis Sentinel द्वारे HA ensured आहे — 3 master + 3 replica nodes + 3 Sentinel monitors.

Redis cluster चे key characteristics: 6 nodes (3 master + 3 replica) across 3 AZs, Sentinel for automatic failover (< 10 seconds), AOF + RDB persistence enabled, maxmemory-policy = allkeys-lru, maxmemory = 8 GB per node, TLS for in-transit encryption, AUTH password for access control. Redis monitoring via redis_exporter → Prometheus → Grafana dashboard with alerts on memory usage, connection count, hit rate, replication lag.

8.2 Use Cases

Use Case	Description	TTL / Notes
API Cache	Hot endpoint responses cached (5-60s TTL)	Permission cache, menu cache, lookup data
Session Store	JWT refresh tokens, OTP attempts, idempotency keys	TTL-based expiry; atomic operations
Rate Limiting	Per-IP + per-user rate limit counters	Sliding window via sorted sets
OTP Cache	OTP codes with 5-min TTL; max 3 attempts	Atomic INCR + EXPIRE for race-condition-free validation
Queue Backend (BullMQ)	Job queue + delayed jobs + repeatable jobs	Persistent across restarts; DLQ for failures
Distributed Locks	Redis Redlock algorithm for cross-pod coordination	Used in scheduled jobs to prevent duplicates
Real-time Pub/Sub	WebSocket fan-out across pods via Redis Adapter	Socket.IO Redis Adapter for multi-pod broadcast
Feature Flags	Per-tenant feature flag cache (5-min TTL)	Refreshed from Postgres on cache miss

Use Case	Description	TTL / Notes
Idempotency Store	Request fingerprint → response cache (24-hour TTL)	Prevents duplicate writes on retry
Geo Indexing	Transport vehicle live location via Redis GEO	Haversine queries for nearest vehicle

8.3 Redis HA Setup

Redis Cluster Topology:

- +-- redis-0 (Master, AZ-a)
- +-- redis-1 (Master, AZ-b)
- +-- redis-2 (Master, AZ-c)
- +-- redis-3 (Replica of redis-0, AZ-b)
- +-- redis-4 (Replica of redis-1, AZ-c)
- +-- redis-5 (Replica of redis-2, AZ-a)
- +-- sentinel-0 (Sentinel, AZ-a)
- +-- sentinel-1 (Sentinel, AZ-b)
- +-- sentinel-2 (Sentinel, AZ-c)

Failover Behavior:

1. Sentinel monitors Master nodes every 1s
2. Master unresponsive for 30s -> mark as down
3. Sentinels elect new Master via quorum
4. Replica promoted to Master
5. Clients re-connect via Sentinel discovery
6. Total failover time < 10 seconds

Persistence:

- AOF (appendonly.aof) every 1s (fsync)
- RDB snapshot every 6 hours
- Backups to S3 every 6 hours (7-day retention)

8.4 Logical Database Allocation

Redis मध्ये 16 logical databases (db 0-15) supported आहेत. PreOne च्या convention प्रमाणे each use case चा separate database आहे ज्यामुळे isolation + monitoring clear होते:

- db 0 — API Cache (hot endpoint responses)
- db 1 — Session Store (JWT refresh tokens, OTP attempts)
- db 2 — Rate Limiting (per-IP + per-user counters)
- db 3 — OTP Cache (mobile → OTP mapping with TTL)
- db 4 — BullMQ Queues (job queues per domain)
- db 5 — Distributed Locks (Redlock + scheduled jobs)

- db 6 — Pub/Sub (WebSocket fan-out via Redis Adapter)
- db 7 — Feature Flags (per-tenant config cache)
- db 8 — Idempotency Store (request fingerprint → response)
- db 9 — Geo Indexing (transport vehicle live location)

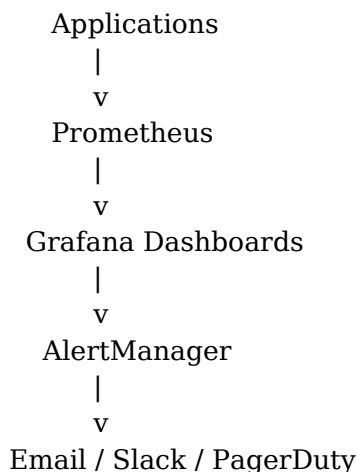
9. Monitoring

9.1 Monitoring Architecture

PreOne ची monitoring stack Prometheus + Grafana + AlertManager द्वारे powered आहे. Monitoring हे observability चा first pillar आहे — 'what is happening' answer करते. Prometheus pull-based model वापरतो ज्यामुळे applications metrics expose करतात (/metrics endpoint) आणि Prometheus त्यांना periodically scrape करतो. Grafana dashboards metrics visualize करतात आणि AlertManager alerts route करतो (Email, Slack, PagerDuty).

Monitoring stack architecture: Applications expose /metrics endpoint via prom-client (Node.js library) → Prometheus scrapes every 15s → Prometheus stores in time-series DB (15-day retention) → Grafana queries via PromQL → AlertManager evaluates alert rules every 30s → routes to on-call engineer via PagerDuty + Slack #engineering-alerts. SLO burn rate tracking द्वारे error budget consumption monitored होतो — high burn rate trigger करतो incident review.

9.2 Monitoring Pipeline



9.3 Monitored Metrics

Metric	Scope	Source	Alert Threshold
CPU / Memory	Container + pod + node level	node-exporter + cAdvisor	Alert if CPU > 80% for 5 min
API Latency	p50, p90, p99 per endpoint	Prometheus histogram	Alert if p99 > 1s for 5 min
Error Rate	5xx responses per second	Prometheus counter	Alert if > 1% for 2 min
Database Connections	Active + idle connections per pool	pgbouncer_exporter	Alert if > 80% pool capacity
Redis Health	Memory usage, connected clients, ops/sec	redis_exporter	Alert if memory > 80%
Queue Length	BullMQ queue depth per queue	bull-board + custom exporter	Alert if > 1000 pending for 5 min
Pod Restarts	Restart count per pod	kube-state-metrics	Alert if > 3 restarts in 10 min
Storage Usage	PVC + node disk usage	kubelet metrics	Alert if PVC > 85%
Replication Lag	Postgres + Redis replica lag	Built-in metrics	Alert if lag > 5 seconds
HTTP Status Distribution	2xx / 4xx / 5xx counts per service	NGINX exporter	Anomaly detection on 5xx spike
Business Metrics	Admissions submitted, fees collected, attendance marked	Custom Prometheus gauges	Dashboard-only; alert via Grafana thresholds
SLO Burn Rate	Error budget consumption per SLO	Derived metric	Alert if burn rate > 2x normal

9.4 Sample Prometheus Alert Rules

```
# prometheus-rules.yaml
groups:
- name: preone-alerts
  rules:
  - alert: HighCPUUsage
```

```

    expr: |
      100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}
[5m])) * 100) > 80
    for: 5m
    labels:
      severity: warning
      team: devops
    annotations:
      summary: 'High CPU usage on {{ $labels.instance }}'
      description: 'CPU usage > 80% for 5 minutes'

- alert: HighAPIErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[2m])) by(service)
    / sum(rate(http_requests_total[2m])) by(service) > 0.01
  for: 2m
  labels:
    severity: critical
    team: backend
  annotations:
    summary: 'High error rate on {{ $labels.service }}'
    description: '5xx error rate > 1% for 2 minutes'

- alert: HighAPILatency
  expr: |
    histogram_quantile(0.99,
sum(rate(http_request_duration_seconds_bucket[5m]))
  by(le, service)) > 1
  for: 5m
  labels:
    severity: warning
    team: backend
  annotations:
    summary: 'High p99 latency on {{ $labels.service }}'
    description: 'p99 latency > 1s for 5 minutes'

- alert: PostgresReplicationLag
  expr: pg_replication_lag_seconds > 5
  for: 2m
  labels:
    severity: critical
    team: devops
  annotations:
    summary: 'PostgreSQL replication lag > 5s'

- alert: RedisMemoryHigh
  expr: redis_memory_used_bytes / redis_memory_max_bytes > 0.8
  for: 5m
  labels:
    severity: warning

```

team: devops
annotations:
summary: 'Redis memory usage > 80%'

- alert: PodRestartLoop
expr: increase(kube_pod_container_status_restarts_total[10m]) > 3
for: 1m
labels:
severity: warning
team: devops
annotations:
summary: 'Pod {{ \$labels.pod }} restarting frequently'
- alert: QueueBacklog
expr: bullmq_pending_jobs > 1000
for: 5m
labels:
severity: warning
team: backend
annotations:
summary: 'BullMQ queue {{ \$labels.queue }} backlog > 1000'

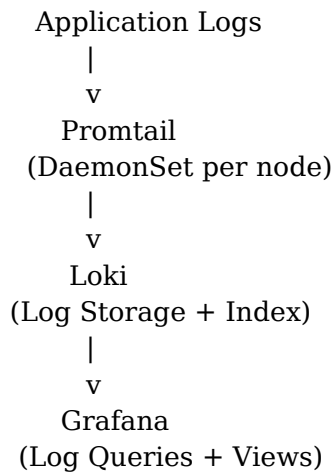
10. Logging

10.1 Logging Architecture

Logging हे observability चा second pillar आहे — 'why is it happening' answer करते. PreOne ची logging stack Loki + Promtail + Grafana द्वारे powered आहे. Loki हे Grafana Labs चे horizontally-scalable log aggregation system आहे जे Elasticsearch/Loki comparison मध्ये cost-effective आहे कारण ते फक्त metadata indexes करतो (न की full-text index). Promtail हे log shipping agent आहे जो DaemonSet म्हणून सर्व nodes वर deploy झालेला आहे आणि container logs scrape करतो.

Application logs Pino (Node.js JSON logger) द्वारे emitted होतात — stdout कडे write होतात. Promtail container logs /var/log/pods/*.log पासून scrape करतो, JSON parse करतो, traceId + tenantId + userId जसे labels extract करतो, आणि Loki कडे push करतो. Grafana Loki queries via LogQL (PromQL-like syntax) द्वारे support करतो. Dashboards pre-built आहेत per service (API, Web, Worker) आणि per log type (requests, audit, errors, jobs).

10.2 Logging Pipeline



10.3 Log Types

Log Type	Content	When Emitted	Format
API Requests	HTTP method, path, status, latency, userId, traceId	Per-request log emitted by NestJS interceptor	Pino JSON format
Authentication	Login attempts, OTP send, token refresh, logout	Always logged with userId + IP + userAgent	Security audit trail

Log Type	Content	When Emitted	Format
Audit Logs	Entity changes (before/after diff) for sensitive operations	Stored in DB + shipped to Loki	Compliance requirement
Background Jobs	Job start, success, failure, retry, DLQ	Per-job log with jobId + queueName + duration	BullMQ instrumentation
Exceptions	Unhandled errors with stack trace + context	Pino + Sentry integration	Alert to Slack #engineering-alerts
Infrastructure Events	Pod starts/stops, HPA scaling, failovers	Kubernetes event stream → Loki	Via kubernetes-event-exporter
WebSocket Connections	Connect, disconnect, room joins, message count	Per-connection log with socketId + userId	Socket.IO middleware
Database Slow Queries	Queries > 500ms with query + duration	Postgres log_min_duration_statement	Tuning + anomaly detection
Cache Hit/Miss	Cache hit ratio per cache key prefix	Redis MONITOR (sampled)	Performance tuning
Deployment Events	Image deployed, version, who triggered, smoke test result	GitHub Actions + ArgoCD notifications	Audit + compliance

10.4 Structured Log Example

```
{
  "level": "info",
  "time": 1736832000000,
  "msg": "Request completed",
  "traceId": "550e8400-e29b-41d4-a716-446655440000",
  "spanId": "a1b2c3d4e5f6",
  "method": "POST",
  "path": "/api/v1/students",
  "status": 201,
  "durationMs": 145,
  "userId": "usr_01HXY...",
  "tenantId": "ten_01HXY...",
  "branchId": "br_01HXY...",
  "ip": "192.168.1.100",
```

```
"userAgent": "Mozilla/5.0...",  
"requestId": "req_01HXY...",  
"service": "preone-api",  
"version": "1.0.0",  
"hostname": "preone-api-7f8b9c6d4-x2y3z"  
}
```

LogQL query examples:

1. All 5xx errors in last hour

```
{service="preone-api"} | json | level="error" | status >= 500
```

2. Slow requests (>500ms) for specific tenant

```
{service="preone-api", tenantId="ten_01HXY..."} | json | durationMs > 500
```

3. Count errors per service in last 5 minutes

```
sum by (service) (count_over_time({level="error"}[5m]))
```

4. Audit logs for specific user

```
{logType="audit", userId="usr_01HXY..."} | json
```

11. Backup Strategy

11.1 Backup Philosophy

Backups हे PreOne च्या disaster recovery चा foundation आहेत. Backup strategy खालील principles follow करते: 3-2-1 rule (3 copies, 2 different media, 1 offsite), automated (no manual backups), tested (daily restore tests), encrypted (at rest + in transit), versioned (multiple point-in-time restores), आणि documented (runbook for every backup type). PreOne च्या backup stack मध्ये Velero (K8s resources + PVs), pgBackRest (Postgres), Redis BGSAVE (Redis), S3 cross-region replication (object storage), आणि GitHub mirror (source code) included आहेत.

Backup verification critical आहे — untested backup हा no backup च समान आहे. PreOne मध्ये daily automated restore tests run होतात जे QA cluster मध्ये restore करतात आणि smoke tests execute करतात. Restore test failure trigger करतो PagerDuty alert. Quarterly full DR drill execute होतो जिथे complete production cluster restore simulate केला जातो.

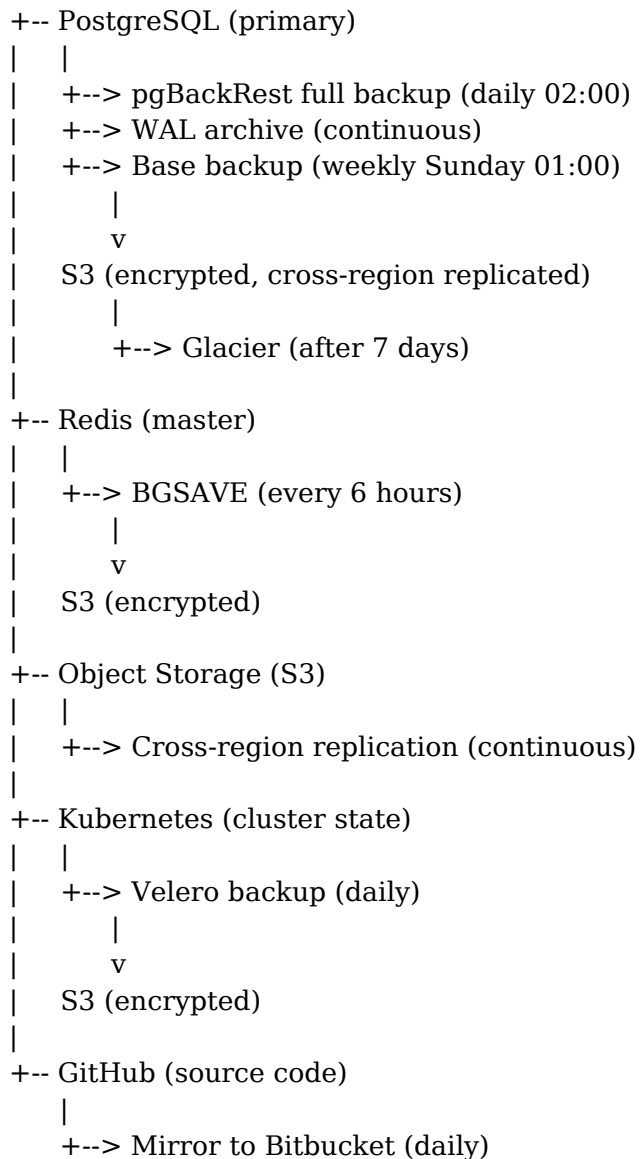
11.2 Backup Schedule

Component	Frequency	Retention	Storage / Notes
PostgreSQL Full	Daily at 02:00 IST	30 Days	pgBackRest → S3 (encrypted); lifecycle to Glacier at 7 days
PostgreSQL WAL	Continuous (every WAL segment)	7 Days	archive_command → S3; enables PITR up to 7 days back
PostgreSQL Base Backup	Weekly (Sunday 01:00 IST)	4 Weeks	Full base backup for faster PITR restoration
Redis Snapshot	Every 6 hours	7 Days	BGSAVE → S3; restore on cold start
Object Storage (S3)	Daily incremental via S3 replication	30 Days	Cross-region replication to DR region
Kubernetes Config	Daily via Velero	30 Days	ConfigMaps, Secrets, Deployments, StatefulSets

Component	Frequency	Retention	Storage / Notes
Kubernetes PVs	Daily via Velero + Restic	30 Days	File-system-level backup for PVs
GitHub Repository	Daily mirror to Bitbucket	Indefinite	Disaster recovery for source code
Container Images	GHCR + replicate to ECR (DR region)	Indefinite	Image retention policy: keep last 100 per tag
Grafana Dashboards	Daily export to Git	Indefinite	Dashboards-as-code via Grafana Operator

11.3 Backup Architecture

Backup Sources:



Verification:

- +-- Daily restore test (automated, QA cluster)
- +-- Quarterly DR drill (manual, full simulation)
- +-- Monthly DR report (RPO/RTO actual vs target)

12. Disaster Recovery

12.1 DR Strategy

Disaster Recovery (DR) हे PreOne च्या business continuity चा critical component आहे. DR strategy दोन pillars वर built आहे: RPO (Recovery Point Objective — max acceptable data loss) आणि RTO (Recovery Time Objective — max acceptable downtime). PreOne चे targets: RPO $\leq$ 15 minutes, RTO $\leq$ 60 minutes. हे targets aggressive आहेत पण preschool domain मध्ये parent communication + fee operations critical असल्यामुळे justified आहेत.

DR architecture multi-layered आहे. Layer 1: Multi-AZ deployment (3 AZs) — single AZ failure handle करते. Layer 2: Automated failover (Patroni for Postgres, Sentinel for Redis) — single node failure handle करते. Layer 3: Cross-region replication (S3, ECR) — region failure handle करते. Layer 4: Multi-region standby (v1.0: active-passive; v2.0: active-active) — complete region failure handle करते. Each layer ची testing regular असते — daily automated tests + quarterly full DR drills.

12.2 DR Metrics

Metric	Target	Implementation
RPO (Recovery Point Objective)	$\leq$ 15 Minutes	Max acceptable data loss; achieved via WAL streaming
RTO (Recovery Time Objective)	$\leq$ 60 Minutes	Max acceptable downtime; achieved via automated failover
Database Recovery	Automatic Failover	Patroni + etcd leader election; < 30 seconds for primary failover
Region Recovery	Restore from Backup	Cross-region restore from S3 + ECR; < 60 minutes for full region failover
Backup Verification	Daily Restore Test	Automated restore to QA cluster; smoke test verification
DR Drill	Quarterly	Full DR simulation; documented runbook; sign-off required

Metric	Target	Implementation
Multi-AZ Deployment	Always On	Pods spread across 3 AZs; PVCs on multi-AZ storage
Multi-Region Strategy	Active-Passive (v1.0)	Primary region active; DR region standby with replicated data
Multi-Region Future	Active-Active (v2.0)	Roadmap — geo-routing via Route53; cross-region Postgres via logical replication
Compliance Reporting	Monthly DR Report	RPO/RTO actual vs target; failover history; restore test results

12.3 DR Runbook (High-Level)

Scenario: Primary Region Complete Failure

Phase 1: Detection (0-5 minutes)

- Cloudflare health check fails for 3 consecutive checks
- Alert triggered to on-call engineer via PagerDuty
- On-call acknowledges + starts incident bridge

Phase 2: Decision (5-15 minutes)

- On-call verifies primary region is genuinely down
- On-call escalates to DevOps Lead + Engineering Manager
- Decision: failover to DR region OR wait for primary recovery
- Decision criteria: estimated primary recovery > 30 min -> failover

Phase 3: Failover (15-45 minutes)

- Update Cloudflare DNS to point to DR region load balancer
- Promote DR Postgres replica to primary (via Patroni)
- Scale up DR Kubernetes cluster (min 3 nodes -> 10 nodes)
- Deploy latest Helm charts to DR cluster
- Verify health checks pass

Phase 4: Verification (45-55 minutes)

- Smoke tests against DR endpoints
- Verify data consistency (last 15 min may be lost - RPO)
- Monitor error rates, latency, queue backlog
- Communicate to customers via status page

Phase 5: Stabilization (55-60 minutes)

- Traffic fully routed to DR region

- On-call hands off to follow-the-sun team
- Post-incident review scheduled within 48 hours

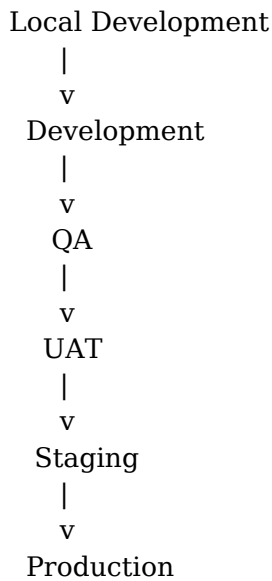
Phase 6: Recovery (post-incident)

- Primary region restored from latest backup
- Data reconciled (DR primary -> restored primary)
- Traffic routed back to primary region
- DR cluster scaled back down to standby

13. Environment Strategy

13.1 Environment Pipeline

PreOne च्या 6 environments आहेत ज्यांचा specific purpose + control level आहे. प्रत्येक environment साठी स्वतंत्र Kubernetes namespace आणि configuration वापरली जाते. Environments strict pipeline follow करतात — कोणताही code change local dev पासून production पर्यंत सर्व environments मधून जाणे आवश्यक आहे. हे approach ensure करते की production मध्ये काही break होणार नाही कारण issues lower environments मध्ये catch होतील.



13.2 Environment Catalog

Environment	Cluster / Location	Deployment Trigger	Purpose
Local Development	developer laptop	Docker Compose for dependencies (Postgres, Redis, MinIO)	Each developer has own stack
Development	shared dev cluster namespace	Auto-deploy from develop branch	Shared by all developers for integration testing
QA	dedicated QA cluster	Auto-deploy from main branch after PR merge	QA team executes test cases here
UAT	UAT cluster namespace	Manual deploy from release branch	Customer UAT — pre-staging gate

Environment	Cluster / Location	Deployment Trigger	Purpose
Staging	production-mirror cluster	Manual deploy from release branch after UAT sign-off	Pre-production; load testing; partner integration
Production	production cluster	Manual deploy after staging sign-off + change advisory board approval	Customer-facing; highest controls

13.3 Environment Policies

Environment management खालील policies follow करते:

- Each environment has separate Kubernetes namespace + config
- No shared databases between environments — full isolation
- Secrets per environment (no reuse); rotated independently
- Production access: break-glass only; MFA + audit logged
- Staging must mirror production (same K8s version, same DB version, same configs)
- Data: synthetic data in lower envs; production-like data in staging via anonymization
- Network policies block cross-environment traffic
- Cost optimization: dev/QA clusters scaled down after-hours (20:00-08:00 IST)
- Environment provisioning via Terraform modules (infrastructure-as-code)
- Decommissioning: lower envs rebuilt weekly for hygiene

14. Security

14.1 Security Architecture

PreOne ची security architecture defense-in-depth model वर based आहे — multiple security layers द्वारे protection. Single layer compromise होण्याचे case मध्ये other layers protection provide करतात. Security layers: Cloudflare (edge — WAF + DDoS + bot protection), NGINX (network — TLS + rate limiting + security headers), Kubernetes (cluster — NetworkPolicies + RBAC + Pod Security Standards), Application (code — JWT auth + input validation + RBAC), Database (data — RLS + encryption at rest + audit logs). हे सर्व layers comprehensive security posture provide करतात.

Security operations core activities: vulnerability scanning (Trivy + Snyk + GitLeaks in CI/CD), penetration testing (annual third-party pentest), security training (annual OWASP training for engineers), incident response (documented runbook + on-call rotation), compliance audits (SOC 2 Type II roadmap + monthly internal audits), secrets management (K8s Secrets for v1 + Vault migration in v1.2), image signing (cosign / Sigstore — verify at deploy time), आणि supply chain security (SBOM generation + dependency pinning).

14.2 Security Controls

Control	Implementation	Notes
HTTPS Everywhere	TLS 1.3 enforced; HTTP auto-redirects to HTTPS	No plain-text HTTP endpoints exposed
TLS 1.3	Minimum TLS version; old TLS rejected at LB	Cipher suite: ECDHE-ECDSA-AES256-GCM-SHA384
Kubernetes Secrets	Base64-encoded at rest; etcd encryption enabled	Vault migration roadmap v1.2
JWT Authentication	RS256-signed; 15 min access + 30 day refresh; rotated	JWKS endpoint for key distribution
Network Policies	Default deny; explicit allow per namespace	API → Postgres allowed; API → Internet denied
Image Vulnerability Scanning	Trivy in CI; admission controller (Kyverno) in K8s	Block deploy on CRITICAL vulnerabilities
Role-Based Access Control (RBAC)	K8s RBAC + app-level RBAC	Least-privilege; service accounts per namespace

Control	Implementation	Notes
Audit Logging	K8s audit log + app audit log	Immutable; shipped to Loki + S3
Pod Security Standards	Restricted profile enforced via PSA	No privileged pods; no host namespaces
Secret Rotation	DB credentials rotated quarterly; JWT keys annually	Automated via sealed-secrets + cron jobs
OWASP Top 10 Protection	Helmet middleware + WAF + input validation	CSP, HSTS, X-Frame-Options, SQL injection prevention via Prisma
DDoS Protection	Cloudflare in front; rate limiting at NGINX + Cloudflare	L3/L4 via Cloudflare; L7 via NGINX
Compliance	SOC 2 Type II roadmap; GDPR; India DPDP Act	Annual external audit; monthly internal audit

14.3 NetworkPolicies Sample

```
# Default deny all ingress + egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: api
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
---
# Allow ingress from NGINX Ingress namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-ingress
  namespace: api
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
```

```
- namespaceSelector:
  matchLabels:
    name: ingress
ports:
- protocol: TCP
  port: 3000
---
# Allow egress to Postgres + Redis only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-data
  namespace: api
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          name: postgres
      ports:
      - protocol: TCP
        port: 5432
  - to:
    - namespaceSelector:
        matchLabels:
          name: redis
      ports:
      - protocol: TCP
        port: 6379
  - to:
    - namespaceSelector:
        matchLabels:
          name: monitoring
      ports:
      - protocol: TCP
        port: 9090
```

15. Scalability

15.1 Scalability Strategy

PreOne ची scalability strategy horizontal scaling वर based आहे — more pods + more nodes, न की bigger machines. Horizontal scaling preferred कारण ते granular, cost-effective, आणि stateless-friendly आहे. Stateless API pods द्वारे any pod can serve any request — यामुळे HPA (Horizontal Pod Autoscaler) effectively scale करू शकतो. Cluster Autoscaler pending pods detect करतो आणि automatically new nodes add करतो. हे सर्व automated आहे — manual scaling intervention required नाही.

Scalability targets: API 3-30 pods, Web 2-10 pods, Worker 2-15 pods, cluster 3-20 nodes. HPA CPU + memory + custom metrics (request latency, queue depth) द्वारे scaling decisions घेतो. Cluster Autoscaler pending pods detect करतो (pods unschedulable due to resource constraints) आणि new nodes provision करतो. Scale-down द्वारे underutilized nodes automatically removed होतात (after 10 min of low utilization).

15.2 Scalability Tactics

Tactic	Description	Scope
Horizontal Pod Autoscaler (HPA)	Scale pods based on CPU/Memory + custom metrics	API: 3-30 pods; Web: 2-10; Worker: 2-15
Cluster Autoscaler	Add/remove nodes based on pending pods	Min 3 nodes; max 20 nodes per node group
Vertical Pod Autoscaler (VPA)	Auto-adjust CPU/Memory requests based on usage	Used for stateful workloads (Postgres, Redis)
Stateless API Pods	No session in pod; all state in Redis/DB	Any pod can serve any request; enables HPA
Redis-based Distributed Queue	BullMQ workers scale independently of API	Workers consume from queue; auto-scaled by queue depth
Read Replicas for Reporting	Heavy reads routed to Postgres replicas	Reports dashboard uses replica; prevents primary load
CDN for Static Assets	Cloudflare CDN for /static/* paths	1-year immutable cache; fingerprinted filenames

Tactic	Description	Scope
Database Connection Pooling	PgBouncer sidecar; 1000 connections per pool	Prevents connection exhaustion under load
Redis Cluster (future)	Current: Redis Sentinel HA; Roadmap: Redis Cluster for sharding	v1.2 — when memory exceeds 25 GB
Caching Strategy	Multi-layer: browser → CDN → NGINX → Redis → DB	Each layer has appropriate TTL; invalidation by version
Graceful Shutdown	Pods receive SIGTERM; drain connections over 30s	Zero-downtime deployments; no dropped requests
Circuit Breakers	opossum library for downstream service calls	Fail fast; prevent cascading failures

15.3 HPA + Cluster Autoscaler Flow

1. Traffic increases -> API pods CPU > 70%
2. HPA detects high CPU via Prometheus metrics
3. HPA increases replicas (3 -> 5)
4. K8s scheduler tries to schedule 2 new pods
5. If insufficient node capacity -> pods Pending
6. Cluster Autoscaler detects Pending pods
7. Cluster Autoscaler provisions new node (1-2 min)
8. New pods scheduled on new node
9. New pods pass readiness check
10. NGINX routes traffic to new pods
11. CPU usage normalizes

Scale-down flow (reverse):

1. Traffic decreases -> CPU < 30%
2. HPA decreases replicas (5 -> 3)
3. Pods gracefully terminate (SIGTERM)
4. NGINX removes from upstream
5. If node underutilized -> Cluster Autoscaler drains
6. Node removed after 10 min

16. Final Production Stack

16.1 Stack Summary

PreOne ची final production stack खालील table मध्ये summarized आहे. हे सर्व components v1.0 release साठी frozen आहेत आणि LOCKED status च्या अंतर्गत येतात. कोणताही change require करतो new ADR + Architecture Review Board approval. Stack चे design philosophy: open-source first, production-proven, cloud-portable, आणि operational simplicity. ही stack PreOne च्या 14 business domains, 530+ APIs, real-time WebSocket channels, आणि background jobs efficiently handle करू शकते.

Layer	Technology	Role
Source Control	GitHub	Git hosting + PR reviews + Issues
CI/CD	GitHub Actions	Build + test + deploy pipeline
Containers	Docker	Image format + build tool
Orchestration	Kubernetes	Container orchestration
Ingress	NGINX	Reverse proxy + load balancer
Backend	NestJS	API + WebSocket gateway
Web	Next.js	SSR + static site
Mobile	React Native + Expo	iOS + Android apps
Database	PostgreSQL HA	Primary data store
Cache	Redis	Cache + session + queue backend
Queue	BullMQ	Background job processing
Storage	AWS S3 / MinIO	Object storage for files
Monitoring	Prometheus + Grafana	Metrics + dashboards + alerts
Logging	Loki + Promtail	Centralized log aggregation
Tracing	OpenTelemetry + Jaeger	Distributed tracing
Backup	PostgreSQL WAL + S3 Backups	PITR + object storage backups

Layer	Technology	Role
Disaster Recovery	Multi-zone with automated failover	RPO ≤ 15min; RTO ≤ 60min

16.2 Stack Justification

प्रत्येक technology choice carefully evaluated आहे alternatives च्या विरुद्ध. Selection rationale documented आहे खालील table मध्ये — यामुळे future engineers ना decisions च context समजेल आणि ते informed changes करू शकतील. हे सर्व alternatives evaluated होते आणि निवड केलेली technology production-proven + team-familiar + cost-effective होती.

Decision	Choice	Rationale
Docker Swarm vs Kubernetes	Kubernetes chosen	K8s ecosystem richer; better long-term bet
Jenkins vs GitHub Actions	GitHub Actions chosen	Native GitHub integration; YAML-based; no infra to manage
AWS ECS vs Kubernetes	Kubernetes chosen	Portability across cloud providers
Traefik vs NGINX Ingress	NGINX chosen	Production-proven; larger community; better WebSocket support
MongoDB vs PostgreSQL	PostgreSQL chosen	ACID; RLS; relational fits DDD aggregates better
RabbitMQ vs BullMQ (Redis)	BullMQ chosen	Single backend (Redis) for cache + queue; simpler ops
ELK vs Loki	Loki chosen	Lower cost; integrates with Grafana; no separate ES cluster
Datadog vs Prometheus+Grafana	Prometheus chosen	Open source; no vendor lock-in; mature ecosystem
Vault vs K8s Secrets	K8s Secrets for v1; Vault later	Simpler ops for v1; Vault adds complexity
AWS RDS vs Self-managed Postgres	Self-managed chosen	Patroni HA + PgBouncer control; RDS limitations on extensions

17. Appendix A — Decision Drivers

17.1 Key Decision Drivers

हा ADR चे सर्व decisions खालील drivers द्वारे influenced आहेत. हे drivers prioritized आहेत — जेव्हा drivers conflict करतात तेव्हा higher-priority driver wins. हे drivers future ADRs साठी reference सुद्धा serve करतात.

Driver	Specifics	Weight
Operational Maturity	Team familiar with K8s + Docker + GitHub Actions	Reduce learning curve; ship faster
Cost Optimization	Spot instances + HPA + auto-scaling	Pay only for actual usage
Vendor Lock-in Avoidance	K8s portable across cloud providers	AWS now, but migration path to GCP/Azure
Security Posture	Defense-in-depth; least-privilege	OWASP + SOC 2 alignment
Compliance	GDPR + India DPDP Act + future SOC 2	Audit logs + DR + access controls
Developer Experience	Local dev mirrors production	Docker Compose + same images
Observability	Three pillars — metrics, logs, traces	Prometheus + Loki + Jaeger
Reliability	Multi-AZ + automated failover	99.9% uptime target for v1.0

18. Appendix B — Glossary

18.1 Terms

Term	Definition
ADR	Architecture Decision Record — document capturing architectural decisions + rationale
CI/CD	Continuous Integration / Continuous Deployment

Term	Definition
HPA	Horizontal Pod Autoscaler — scales pods based on metrics
PITR	Point-in-Time Recovery — restore database to specific timestamp
RPO	Recovery Point Objective — max acceptable data loss
RTO	Recovery Time Objective — max acceptable downtime
WAL	Write-Ahead Log — Postgres transaction log for durability + replication
PgBouncer	Connection pooler for PostgreSQL
Patroni	HA solution for PostgreSQL using etcd for leader election
StatefulSet	K8s workload type for stateful applications (DB, cache)
ConfigMap	K8s resource for non-confidential configuration data
Secret	K8s resource for sensitive data (passwords, tokens)
Ingress	K8s resource managing external access to services
Namespace	K8s mechanism for isolating groups of resources within a cluster
Prometheus	Pull-based metrics collection + time-series database
Grafana	Visualization + dashboarding tool for metrics + logs
Loki	Horizontally-scalable log aggregation system by Grafana Labs
Promtail	Log shipping agent for Loki (similar to Promtail for Prometheus)
OpenTelemetry	Vendor-neutral observability standard for traces, metrics, logs

Term	Definition
Jaeger	Distributed tracing backend (CNCF project)
Velero	K8s backup + restore tool for cluster resources + PVs
BullMQ	Redis-based queue library for Node.js background jobs
RBAC	Role-Based Access Control
DLQ	Dead Letter Queue — queue for messages that failed processing
CDN	Content Delivery Network — edge caching for static assets

19. Document Control

19.1 Document Information

Document Title	PreOne ADR-111 DevOps & Infrastructure Architecture
ADR ID	ADR-111
Status	LOCKED
Document Version	1.0
Date	2026-07-14
Product	PreOne — Enterprise Preschool Operating System
Predecessor	ADR Catalog v1.0, Backend TD v1.0, Frontend Architecture v1.0
Successor	DevOps Runbook, Kubernetes Manifests, Terraform Modules, Production Release
Scope	Docker, Kubernetes, CI/CD, NGINX, PostgreSQL HA, Redis, Monitoring, Logging, DR
Classification	Internal Engineering Reference
Prepared by	PreOne Architecture & DevOps Team

19.2 Change History

Version	Date	Author	Changes
1.0	2026-07-14	Architecture & DevOps Team	Initial LOCKED version — 16 chapters covering Docker, K8s, CI/CD, NGINX, Postgres HA, Redis, Monitoring, Logging, Backup, DR, Environments, Security, Scalability, Final Stack
0.9	2026-07-10	Architecture & DevOps Team	Pre-lock draft — security review feedback incorporated
0.5	2026-07-01	Architecture & DevOps Team	Initial draft — chapters 1-8 (Docker, K8s, CI/CD, NGINX, Postgres, Redis)
0.1	2026-06-20	Architecture & DevOps Team	Skeleton — TOC + section headers + placeholder content

19.3 Approval Matrix

Role	Name	Responsibility	Status
DevOps Lead	—	Infrastructure architecture, K8s setup, CI/CD	Pending
Engineering Manager	—	Team capacity, on-call rotation, budget	Pending
Security Lead	—	Security review, compliance sign-off	Pending
Architecture Review Board	—	ADR review + LOCKED status approval	Pending

Role	Name	Responsibility	Status
CTO	—	Final approval for production release	Pending

19.4 Next Steps

- Architecture Review Board sign-off (this week)
- Security review for NetworkPolicies + RBAC (next sprint)
- DevOps team walkthrough + Q&A session
- Terraform modules development (infrastructure-as-code)
- Helm charts authoring for all services
- GitHub Actions workflows implementation
- Production cluster provisioning (EKS + node groups)
- PostgreSQL HA setup (Patroni + etcd + PgBouncer)
- Redis Sentinel setup (3 masters + 3 replicas)
- Monitoring stack deployment (Prometheus + Grafana + AlertManager)
- Logging stack deployment (Loki + Promtail + Grafana)
- Backup + DR verification (daily restore tests)
- Production Runbook documentation
- Quarterly DR drill scheduling